

AI-POWERED MOCK INTERVIEW PLATFORM

Name	Enrolment No.
Priyanshi Rana	9922103265
Suhani Raj	9922103129
Dhruv Agarwal	9922103261

Name of Supervisor: Dr. Rashmi Kushwah



April 2026

**Submitted in partial fulfillment of the Degree of
Bachelor of Technology**

in

**Computer Science Engineering / Information Technology
Computer Science Engineering**

**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING &
INFORMATION TECHNOLOGY
JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY, NOIDA**

CERTIFICATE

This is to certify that work submitted under the title “AI-Powered Mock Interview Platform” by Priyanshi Rana (9922103265), Suhani Raj (9922103129), Dhruv Agarwal (9922103261) in part fulfilment of the requirement for the award of the degree of Bachelor of Technology in Computer Science Engineering of Jaypee Institute of Information Technology, Noida has been done by them under my direction and supervision. I declare that the work has not been submitted anywhere else for any other degree or for any other diploma from any other University or Institution. Students have good analysis and are technically sound.

Signature of Supervisor _____

Name **Dr. Rashmi Kushwah**

Designation **Assistant Professor (Sr. Grade)**

Department Computer Science & Information Technology

Date , 2026

DECLARATION

We affirm that this submission is our own work and that, to the best of our knowledge and belief, it contains no portion which has been previously published or written by another person nor any portion which has been accepted for the award of any other degree or diploma awarded by a university or other institute of higher learning, but where appropriate acknowledgement has been given in the text.

Place: Noida

Date:

Name: PRIYANSHI RANA

Enrolment No.: 9922103265

Name: Suhani Raj

Enrolment No: 9922103129

Name: DHRUV AGARWAL

Enrolment No.: 9922103261

ACKNOWLEDGEMENT

We would like to thank all of those who have helped us to complete this project. We would like to thank our project supervisor Dr. Rashmi Kushwah very sincerely for her brilliant guidance, support and assistance throughout our research. Her guidance has helped us to embellish our project in many aspects and her precious suggestions helped us to bypass many technical problems.

We would also like to thank sincerely Dept. of Computer science & Information Technology of Jaypee Institute of Information Technology for providing us with sufficient resources, infrastructure and academic environment for research & development. We would also like to thank all the faculty members who supported us with their expertise at critical stages of our project development. We would also like to thank our seniors who shared their experience with us and have given us practical suggestions leading to significant improvement in our implementation.

Signature of Students

Priyanshi Rana (9922103265)

Suhani Rai (9922103129)

Dhruv Agarwal (9922103261)

Date:

, 2026

ABSTRACT

Recruiting software engineering candidates increasingly depends on a candidate's ability to think and communicate under time constraints. Conventional interview preparation methods, such as studying textbook, cracking problems on coding sites or practicing mock interviews with friends, rarely capture the time pressure and communication dynamics of a technical interview, and there is a large gap between what a candidate expects to experience and what the candidate actually experiences.

This project is about an AI Interview Agent web-app for voice based interview preparation built with Next.js 15, firestore, Vapi AI and Google Gemini. The platform allows users to simulate an interview scenario with AI interviewer, which conducts a mock interview through voice conversation and gives instant AI generated feedback. Users can see their improvement over the time. Platform has six major features: : an analytics dashboard to keep track of progress, resume upload with AI generated personalized interview questions, timer component with live countdown during interview, live transcript of interview, download of feedback report in PDF and practice mode as text based interview for cases where voice based interview is not possible.

The platform was built using an iterative approach, where each feature module was built, tested and integrated into the final platform independently. Firebase Firestore was used as the primary database for storing user data, interview sessions and feedback. Vapi AI was used to manage real time voice based conversation. Google Gemini was used to generate feedback and personalized interview questions from uploaded resumes.

This platform democratizes interview preparation by making high-quality, AI based mock interviews accessible to any student or job seeker with a web browser, regardless of geography or financial limitations. This platform is great for students from tier-2 and tier-3 cities where access to formal interview coaching is limited.

TABLE OF CONTENTS

Section	Description	Page
List Of Figures/Tables/Abbrevations		1
CHAPTER 1: INTRODUCTION		4
1.1	Background	4
1.2	Problem Definition and Objectives	4
1.3	Social & Environmental Relevance	5
1.4	Project Plan	6
CHAPTER 2: LITERATURE REVIEW		11
2.1	Critical Review	11
2.2	Summary Table	12
2.3	Identification of research gaps	13
CHAPTER 3: SYSTEM DESIGN & METHODOLOGY		15
3.1	Design Considerations	15

3.2	Methodology	15
3.3	Modelling	18
3.4	Tool and Technique Selection	18
CHAPTER 4: IMPLEMENTATION & TESTING		20
4.1	Development Details	20
4.2	Experimental Setup	28
4.3	Test Cases	29
4.4	Safety Measures	32
CHAPTER 5: RESULTS & DISCUSSION		33
5.1	Result analysis	33
5.2	Comparison	35
5.3	Economic Analysis	37
5.4	Comparison with existing solutions and standards available	37

CHAPTER 6: CONCLUSION & FUTURE SCOPE		38
6.1	Summary of Achievements	38
6.2	Impact on society, Environmental sustainability Ethical issues	39
6.3	Limitations	39
6.4	Future Work	40
REFERENCE & APPENDICES		41

List of Figures

Chapter 3:

- **Fig. 3.1 Architecture of Interview Platform**
- **Fig. 3.2 Use Case Diagram of Interview Platform**
- **Fig. 3.3 System Flow Diagram**

Chapter 4:

- **Fig. 4.1 Sign-in/Sign-up page**
- **Fig. 4.2 Home page**
- **Fig. 4.3 Resume Upload page**
- **Fig.4.4 AI based question generation**
- **Fig. 4.5 Voice Input and live transcript**
- **Fig. 4.6Feedback page**
- **Fig. 4.7 Use Case Diagram**
- **Fig. 4.8 Class Diagram**

List of Tables

Chapter 1

- **Table 1.1 Work breakdown structure**
- **Table 1.2 Timeline**

- **Table 1.3 Challenges and Risk Management**

Chapter 2

- **Table 2.1 Literature Summary Table**
- **Table 2.2 Model evaluation table**

Chapter 4

- **Table 4.1 Tools and Technologies Used**
- **Table 4.2 Login Module Testing**
- **Table 4.3 Resume Upload Module Testing**
- **Table 4.4 Voice Input and Transcript Testing**
- **Table 4.5 Feedback Generation Testing**
- **Table 4.6 Dashboard and Analytics Testing**
- **Table 4.7 PDF Report Generation Testing**
- **Table 4.8 PDF Report Generation Testing**

Chapter 5

- **Table 5.1 Performance Summary Table**
- **Table 5.2 Interview System Performance Result**
- **Table 5.3 Interview Feature Performance**
- **Table 5.4 Component vs Response Time**

- **Table 5.5 Comparison with Objectives**
- **Table 5.6 Bill of Materials**
- **Table 5.7 Resource Utilization**
- **Table 5.8 Comparison with Existing Systems**

List of Abbreviations

1. **AI – Artificial Intelligence**
2. **NLP – Natural Language Processing**
3. **API – Application Programming Interface**
4. **UI – User Interface**
5. **UX – User Experience**
6. **PDF– Portable Document Format**
7. **DB – Database**
8. **SSR– Server Side Rendering**
9. **TS– TypeScript**
10. **HTTP – HyperText Transfer Protocol**

CHAPTER 1: INTRODUCTION

1.1 Background

In the current job market, especially software engineering and technology related ones, technical interview is a prerequisite skill that decides one's career. Students and fresh graduates spend significant amount of time on self-learning, practicing coding problems on leetcode, and reading about data structures. But few get feedback based realistic mock interview experience before actual company interviews.

Moreover, human led mock interview is expensive, professional interview coaching is at cost of few thousand rupees per session, and peer to peer mock interview is not structured question answering session, i.e. professional tone and proper question answering session and structured question answering session, and merit based evaluation is difficult to be done by a peer. Also, students in Tier-2 and Tier-3 cities don't have many networking opportunities to find an experienced mentor for mock interview practice.

Recently, with the advent of conversational AI technologies , for example large language models and real-time voice synthesis APIs , we can build a fully automated voice driven mock interview system that behaves like human interviewers. Vapi AI provides a real-time voice conversation SDK, and Google Gemini offers powerful reasoning and feedback generation capabilities. Together, these technologies enable building a free, always available, cost free interview practice platform that any student can access from a browser.

1.2 Problem Definition and Objectives

The big problem that this project will address is the fact that there isn't a readily accessible realistic simulation of a voice interview that provides structured responses and progress tracking for the person trying to obtain a position in software engineering. Pramp, Interviewing.io, and Big Interview all require a human to be available to interview, charge a subscription fee, or are text-based instead of using real-time voice AI. None combine resume-based question personalization with real-time voice AI and structured AI-generated feedback

in a single free-to-use web app. To develop a real-time voice-based interview simulation system using Vapi AI, which facilitates mock interviews using an AI interviewer that speaks naturally, and can ask both technical and behavioral questions for software engineering domains.

To include Google Gemini AI for generating elaborate post-interview feedback that scores the candidate on their clarity of communication, the technical accuracy of their responses, their problem-solving approach, confidence level, and also provide specific recommendations for improvement. To include a resume analysis feature where the user can upload their PDF resume, and Gemini AI parses the content and generates interview questions related to user's stated skills, projects and experience.

1.3 Social & Environmental Relevance

This platform has a strong social relevance in the Indian education & employment ecosystem. Placement success rate in non-IIT & NIT colleges is considerably lower because students in these colleges do not have access to high-quality interview preparation materials. This platform is helping to close this opportunity gap by giving any student free access to a AI-powered interview simulation tool that can be accessed from any device with a browser.

Practice Mode , the text-based Q&A module that does not rely on the voice engine, is another key accessibility feature that allows students with limited bandwidth, older devices and speaking accessibility needs to still access the powerful preparation features. The resume-based question generation feature also ensures that the experience is tailored to the student and its not just a one-size-fits-all question bank for students with varied academic backgrounds.

From an environmental perspective the platform provides a paperless interview preparation model. Training centers print study material, have physical infrastructure and require commute. All these environmental costs are avoided because the platform is web based. Everything is accessed by a browser, the back end is running on Vercel's edge network and all study material, feedback and reports are digital.

1.4 Project Plan

1.4.1 Work Breakdown Structure

This work consists of eleven main functional modules. These modules were independently developed and integrated in a single platform:

S.NO.	MODULE	DESCRIPTION
1.	Requirement Analysis	Functional & non functional requirement identification, reviewing similar platforms.
2.	Technology Stack Setup	Setting up Next.js 15, Firebase, Vapi AI, Google Gemini SDK, Tailwind CSS
3.	Authentication Module	Google OAuth login, session management, user profile storage in Firestore
4.	Core Interview Engine	Vapi AI voice inout, real time conversations, question logic
5.	AI Feedback System	Google Gemini API post interview scoring and detailed feedback generation
6.	Analytics Dashboard	Charts for performance monitoring per category and over time
7.	Resume Upload & AI Analysis	PDF Resume parsing, Gemini generates custom interview questions
8.	Interview Timer Component	Countdown timer with progress bar integration during live interview sessions
9.	Live Transcript Display	Real-time voice-to-text transcript panel rendered during interview

10.	Export Feedback as PDF	Downloadable PDF report generation for completed interview sessions
11.	Practice Mode Page	Text-based Q&A without full voice agent for low-bandwidth/offline practice

Table 1.1 Work breakdown structure

1.4.2 Timeline

Phase	Activities	Duration
Phase1:Planning	Requirement gathering, finalisation of tech stack	Week 1 – Week 2
Phase 2: Setup	Project scaffolding, Firebase & Vapi configuration	Week 2 – Week 3
Phase 3: Core Dev	Auth, Interview engine, Gemini AI feedback	Week 3 – Week 7
Phase 4: Feature Dev	Analytics, Resume AI, Timer, Transcript, PDF Export, Practice Mode	Week 7 – Week 11
Phase 5: Testing & Deployment	Bug fixes, Vercel deployment, documentation	Week 11 – Week 12

Table 1.2 Timeline

1.4.3 Roles and responsibilities

Suhani (9922103129): I built the application from scratch using Next.js 15 with App Router pattern. I integrated and configured Firebase Authentication for secure login with Google OAuth, set up the Firestore schema with the right rules and data models to store user profiles, interview sessions, feedback records etc. I also integrated and used the Vapi AI SDK to control

real time voice conversation during mock-interview sessions, handling microphone permissions, session state etc.. In order to build the AI features of the platform I integrated Google Gemini API to build 2 core features. First was the post-interview level of feedback that I built and refined the prompts to ask Gemini to score a candidate on a range of parameters like communication clarity, technical correctness, problem solving approach, confidence etc. and return a structured and actionable feedback for each interview session. Second was the feature to allow users to upload a PDF resume that I built upon to extract text from the PDF and pass it to Gemini with a set of prompts that would generate interview questions based on the skills, projects and work experience listed in the resume. I also maintained the GitHub repository.

Priyanshi (9922103265): worked on the backend development of the project. She built the server side API routes in Next.js that acted as a bridge between the front end and external services to validate the incoming requests, handle errors and parse the responses to ensure the data is clean and consumable in the application. On the feature side she built the Analytics Dashboard where category-wise and time-series performance charts were fetched from Firestore and visualized a candidate's progress in all the interview sessions. She also built the Interview Timer Component where a live countdown timer with a progress bar is shown in the interview session page so that the candidates can manage their time in each session.

Dhruv (9922103261): worked on the frontend development and QA on the project. He styled the UI using Tailwind CSS and ui components to give the application a clean, polished, and responsive look across all pages. In regards to features, he built the Live Transcript Display that listened to Vapi events and appended transcript segments to the UI in realtime in order to display real-time speech-to-text output during interviews. He built the PDF Export that exported a formatted version of interview feedback to a downloadable PDF. He built the Practice Mode page which included a text-based Q&A interface for users who could not use the Voice Agent. Dhruv also led the overall testing effort by writing and running end-to-end tests across the application to reduce the chance of bugs finding their way into production and to ensure that all changes worked as intended on different browsers and screen sizes.

1.4.4 Challenges and risk management

Challenge	Risk Level	Mitigation Strategy
Vapi AI API limit exceeded	High	Rate limiting on calls, usage monitoring, fallback to text-based practice mode
Firebase quota exhaustion	Medium	Pagination and Firestore read optimization, caching frequent queries
Gemini API response latency	Medium	Async feedback generation, loading indicators, retry mechanism
Browser microphone permission denied	High	User guidance prompts, Practice Mode as fallback without voice dependency

Table 1.3 Challenges and risk management

1.5 Scope and Limitations

The scope of this project is the whole development of a full-stack web application for AI interview preparation. The following features are in scope:

- Voice based interview session on software engineering topics such as Data Structures & Algorithms, System Design, Object-Oriented Programming and Behavioral questions.
- Login using Google OAuth via Firebase and keeping user data isolated in Firestore.
- AI generated post-interview feedback on communication, technical correctness, confidence, problem solving, stored per user and per session of interview.

- Performance analytics based dashboard with category-wise and time-series charts based on historical data of sessions.
- PDF upload of resume and Gemini AI powered extraction of skills and projects for generating interview questions contextually.
- Export the entire feedback report as a single PDF with one click of a button for any completed interview session.

The following features are not included in the scope of this project:

- This platform does not have real coding assessment capabilities like code editor/automated test cases, it focuses on verbal and conceptual interview preparation.
- This platform does not support non-english interview sessions in the current version. The multi-language support is not included in this project as Vapi AI is optimized for english.
- This platform does not have a peer-matching system for human to human mock interview sessions, all the mock interview sessions are with the AI agent.
- This platform does not have any job listings, curated question banks specific to companies or job portal integrations such as LinkedIn or Naukri.
- The platform does not support offline functionality. All the features require internet connectivity as we have used Firebase, Vapi AI and Gemini API.
- The Practice Mode does not have voice feedback and scoring right now, it is only text based Q&A.

CHAPTER 2: LITERATURE REVIEW

2.1 Critical Review

Artificial intelligence based interview preparation and assessment systems have become popular recently. It provides interactive human-machine conversation. Some research has been conducted on interview preparation and assessment system for interviewees.

Interview preparation system using artificial intelligence, The recent interview preparation system used Natural Language Processing to generate context-sensitive questions dynamically depending on the answers. It can also assess the user and provide the user with a feedback . This is better than the traditional static interview system . There are some limitations of the current interview preparation system such as generating follow-up question and adapt the question to the level of user skill .

AI chatbot, chatbots can be used as a conversational agent to simulate human conversation using machine learning . They have some limitations in the application such as interpreting a wide variety of input and maintaining coherency in a conversation . They can be used in many fields such as customer service and health care .

Interview chatbot using neural network, Interview chatbot using neural network can be trained using a dataset of questions and answers. The model can be used to generate questions and to answer user answers . The system can be developed through the data collection, training and deployment. The quality of the system depends on the data it is trained on .

Another significant paper on conversational AI interview suggests that the AI interview can engage in structured conversations and extract relevant insights, but at times can be erratic, omit topics or be biased in responses .

In general, the current literature suggests that AI interview systems can play a role in practice and assessment, but there is still room for improvement in personalization, real-time response and accuracy.

2.2 Summary Table

Paper	Methodology	Approach	Advantages	Limitations
AI Interview Preparation Bot (2025)	NLP-based conversational AI	Context-aware question generation	Real-time interaction, adaptive questions	Limited follow-up depth
Chatbot Survey (2022)	Machine Learning + NLP	General conversational datasets	Wide applicability, automation	Poor context understanding
Neural Network Interview Chatbot	Neural networks	Training datasets from interview Q&A	Improved interaction quality	Data dependent
SmartHire AI Platform	AI + feedback system	Real-time user responses	Performance evaluation, scalable	Limited emotional analysis
AI Interview Systems Review	Multiple approaches (rule-based, AI)	Mixed datasets	Comprehensive system design	Lack of standard benchmarks
AI Interviewer Study	Conversational AI	Human interaction data	Structured interviews, scalable	Unnatural behavior, bias

Table 2.1 Literature summary Table

The following table compares the language models evaluated during development:

Model Type	Type	Size	Use Case	Advantages	Limitation	My Observation
AI Interview Bot	NLP-based AI	Medium	Mock interview practice	Generates dynamic questions, interactive	Weak follow-up questions	Good for basic practice but lacks depth
General Chatbot	Rule-based / ML	Small	Basic conversation	Simple and low cost	Cannot understand complex queries	Not suitable for interview preparation
Neural Network Chatbot	Deep Learning	Large	Advanced Q&A interaction	Better accuracy and learning	Requires large dataset	Effective but resource heavy
SmartHire Platform	AI Evaluation System	Large	Full interview simulation	Provides detailed feedback	Expensive and limited personalization	Good but not accessible to all users
Conversational AI Interviewer	Advanced AI	Large	Realistic interview simulation	Human-like interaction	Bias and unnatural responses	Promising but not fully reliable

Table 2.2 Model evaluation table

2.3 Identification of Research Gaps

Comparison of Existing AI Interview System. There are multiple limitations in existing solutions:

- Limitations in Personalisation. Existing solutions do not personalize the question generation and hence final response is not that effective. Most of the solutions are generating generic questions and not utilizing any information about the user background, skill set or resume.
- No Feedback During Interview. Feedback is provided only at the end of interview. No feedback is provided during the interview.
- No Performance Dashboards. Most of the solutions do not have performance dashboards or analytics to track your performance over time and identify your weak areas.
- No Advanced Features like Voice Interaction and Live Transcript. Many solutions do not have voice interaction or live transcript. So the interview experience is not as close to the real interview.
- Do not Understand Human Responses. Most of the AI systems do not understand human responses well to provide meaningful feedback.
- No Downloadable Reports. Very few tools provide downloadable reports or structured feedback.

CHAPTER 3: SYSTEM DESIGN & METHODOLOGY

3.1 Design Considerations

In the development of this AI based interview platform we took into consideration a number of practical factors that would make the system useful, reliable and convenient. In the economic aspect, this system is primarily built using free or low cost services: Firebase and Web respectively. Thus the system can be accessed by students without incurring extra cost. In the environmental aspect, this system promotes the use of a fully digital system. This reduces the use of printed material as well as the coaching sessions. In the social aspect, this is a crucial factor in this project as many students have no access to interview training and guidance. This system provides them with an opportunity to practise their interview at any time and to boost their confidence.

In the ethical aspect, this system will not save personal data that is not needed. Also user data such as responses and resumes need to be handled with care. The system is only intended to help the user improve their performance and not to evaluate the user. In the health and safety aspect, the user can practise in their own comfortable environment with no stress. There is no real interview. This will help to reduce anxiety. The system is constructed in a modular manner so that new updates can be added.

3.2 Methodology

The approach adopted in this project is to simulate a smooth and realistic interview experience to the user. The system is designed in such a manner that the system behaves like a basic interviewer in terms of user interaction to final feedback generation.

Initially user logs into the system using the authentication feature. After authentication the user is provided with an option to select the type of interview he wants to take. In our project we have implemented two modes, resume based interview and direct interview. If user selects resume based interview, user is asked to upload his/her resume. If user selects direct interview mode, the system directly generates general interview questions by skipping the resume step.

Once the interview starts, the system displays the interview questions one by one. The user answers the questions by either speaking or typing, as per the implemented features. In case the user speaks the system will use the browser native speech to text capabilities, and display it as the real time transcript. This feels more engaging and realistic, like a real interview.

Runs a timer in the background, to simulate real interview environment and track the timing of answers. Sends all the answers from the user to the AI module, which will analyze the relevancy, completeness, etc.

At the end of the interview, the AI generates the feedback based on the answers given by the user. The system displays the feedback to the user in a structured manner. The user's performance is also reflected in the analytics dashboard. The user is also allowed to download the feedback in a PDF format.

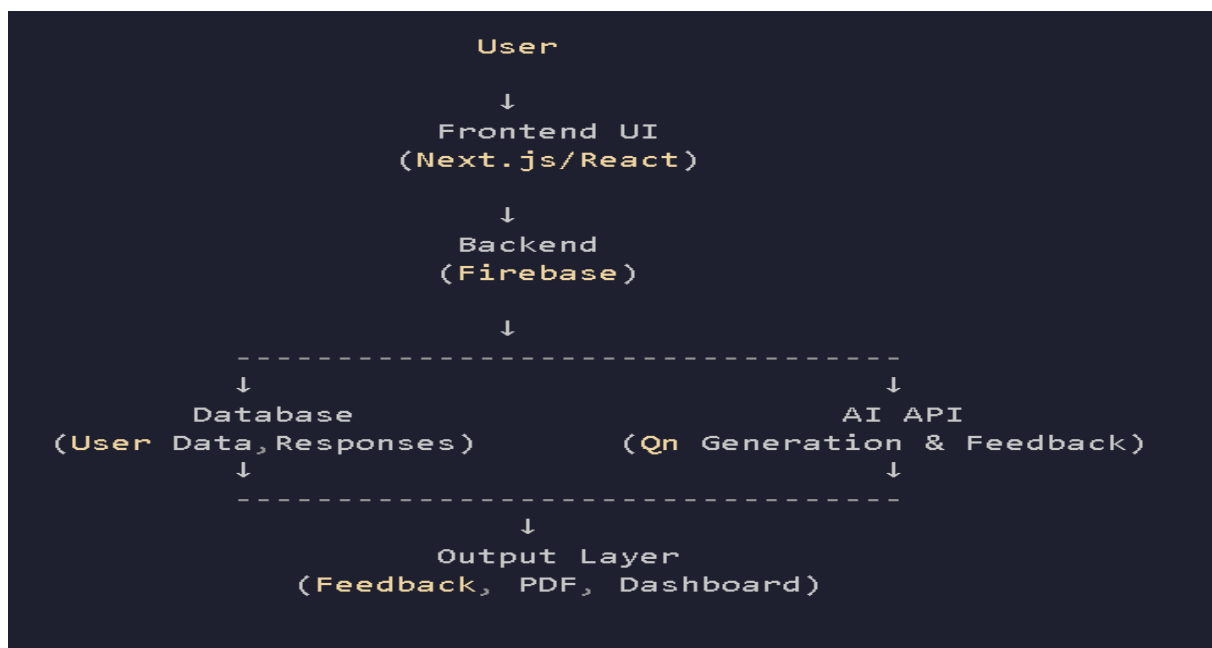


Fig 3.1 Architecture of Interview Platform

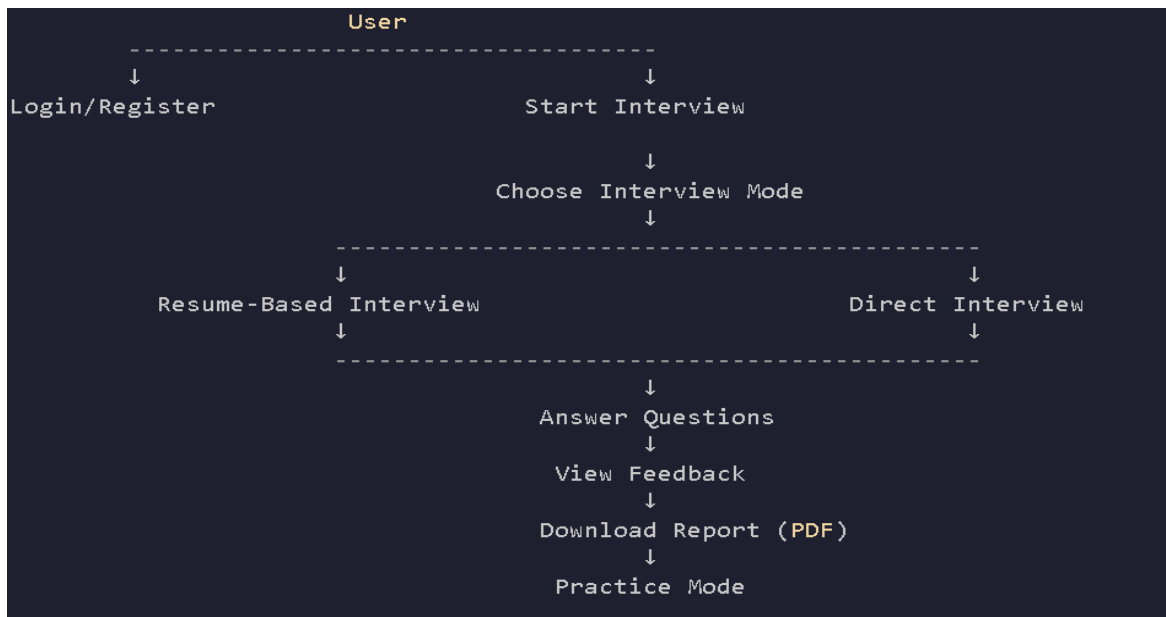


Fig 3.2 Use Case Diagram of Interview Platform

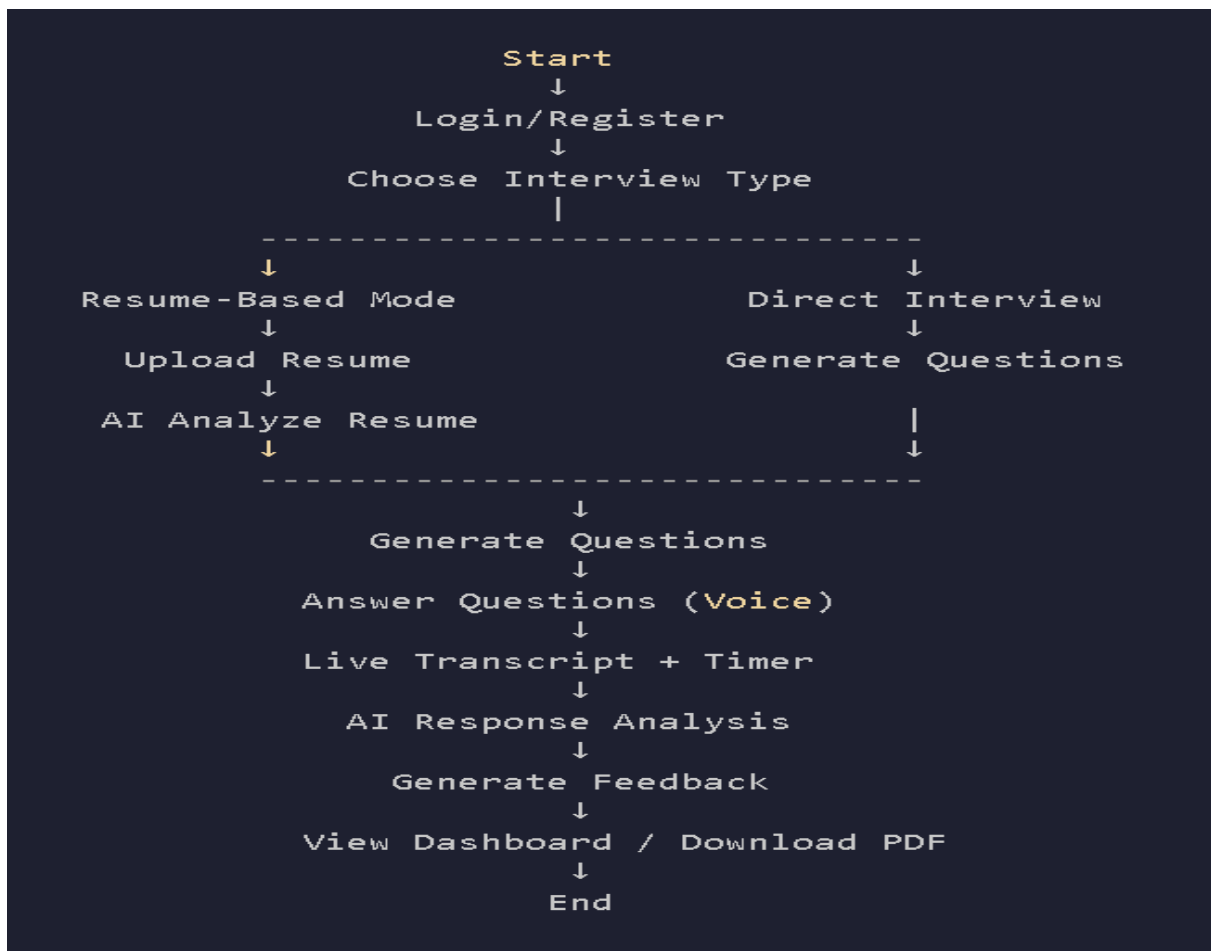


Fig 3.3 System Flow Diagram

3.3 Modelling

In this project, there is no complex mathematical or statistical model but the system has a logical model for execution. The system has a logical model as follows: input, processing, and output. The input is user responses which will be either voice or text as the user will respond to the question in the interview. Finally, the output will be feedback and performance evaluation.

The model will be based on evaluation of responses such as the relevance of the user's response to the question, clarity of expression, and completeness. The system will also be AI-based natural language processing in which the user's response will be analyzed and similar to the expected pattern to generate feedback. There is no specific formula for this, but there is a logic behind the responses and the feedback will be meaningful.

3.4 Tool and Technique Selection

Every tool used in Interview Platform was chosen for a specific reason, not just convenience.

We chose **Next.js (React Framework)** as the frontend main technology because it provides quick and scalable development of user interfaces, and routing and server-side rendering and other features result in better performance and user experience.

We decided to use **Firebase** as backend for all backend services. The main reasons is authentication and database. Firebase helps us to not have to create a server, it is secure and scalable. Firebase Auth takes care of the hassle of our users signing up or logging in, and Firestore easily stores the user, the answers of the interview and the feedback. We considered node.js with express and many others, but the speed and integration with Firebase was better.

We have used public **AI APIs**, not a custom-built model for the AI-based features. Building a model needs massive datasets, huge computational resources and time. Pre-trained APIs are more accurate and more readily available for such real-time applications as interview question generation and success evaluation.

The **browser based Speech Recognition API** was used for voice interaction user voice was transcribed into text. No extra hardware or complicated setup is needed. It is performed in the browser itself and transcription is done in real time which is most suitable for simulating a real interview scenario.

To enable the user to download their interview feedback in a formatted report for downloadable reports we needed to add a **PDF generation library** as creating reports manually is not efficient and consistent.

Tools of programming and version control: **VS Code** and **GitHub**. VS Code and GitHub are used to write code and manage different code versions.

CHAPTER 4: IMPLEMENTATION & TESTING

4.1 Development Details and Design Diagrams

1. User Authentication System

Goal: allow user to login to site and read his/her interview data.

Frontend: User logs in using the login form with email and password.

- The login component receives the user input and passes it for authentication.
- User is logged in and session is created for further access.
- User is redirected to the dashboard upon login.

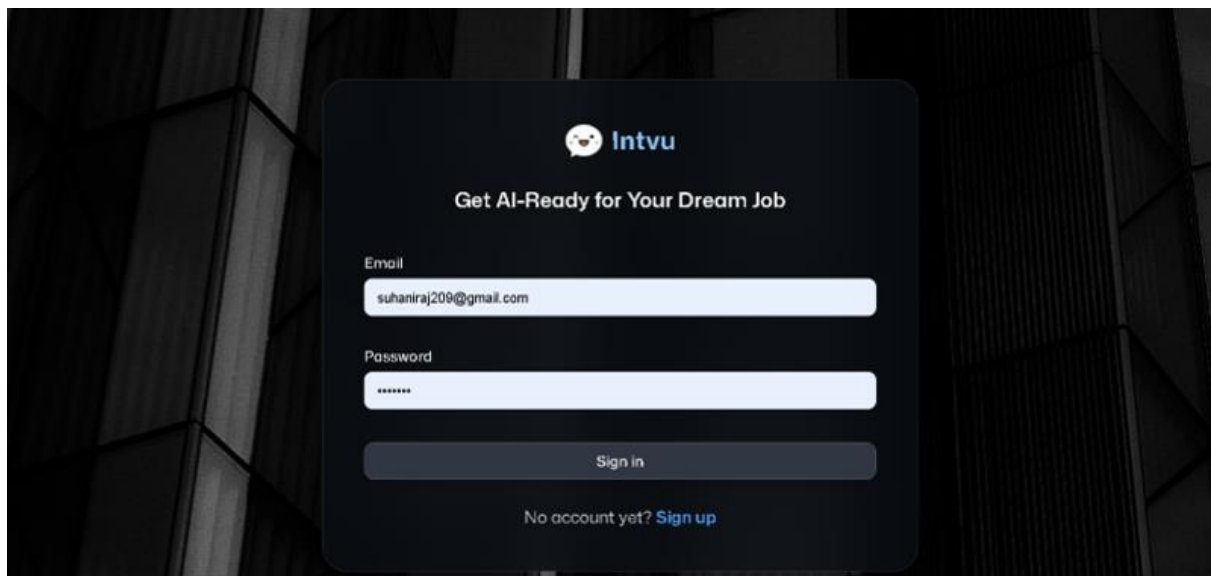


Fig 4.1 Sign-in/Sign-up page

Backend: Firebase Authentication is used to validate the user credentials.

- Authentication request is sent to Firebase services.
- Secure session is created when logged in.
- Error cases for valid login and network error are handled.

2. Interview Mode Selection

Goal: Give users the ability to pick the type of interview so they have some flexibility.

Frontend: The user chooses interview type from the available ones.

- Two options are being provided: Resume based interview and Direct interview.
- Next workflow is based on the chosen option.
- UI shows what was chosen.

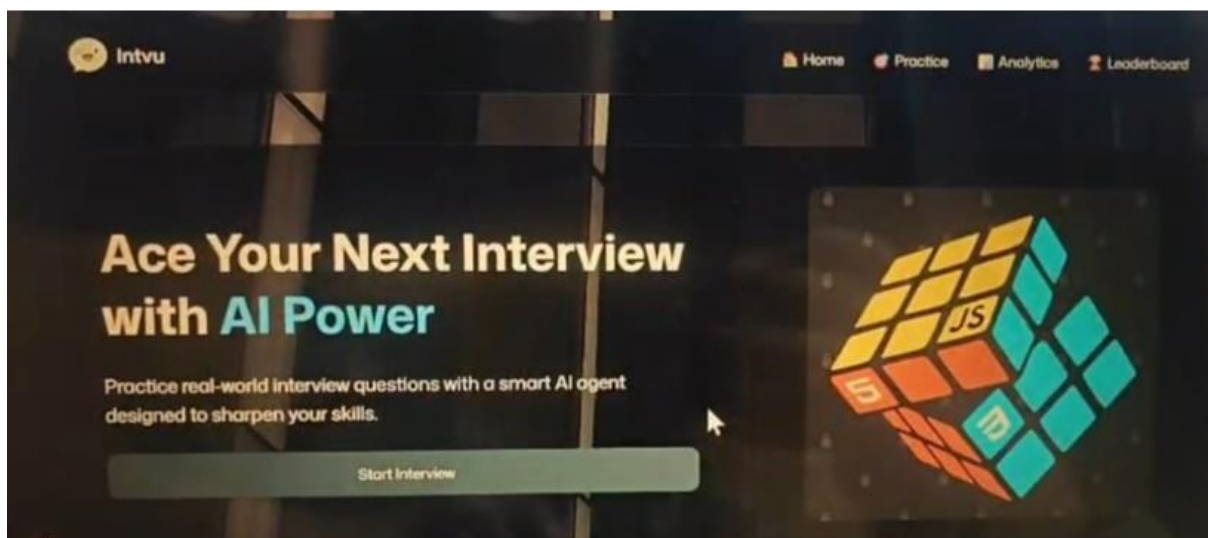


Fig 4.2 Home page/ Dashboard

Backend: System has processes selected mode.

- If resume based mode selected resume upload is enabled.
- If direct mode selected general questions are generated.
- System will route your request.

3. Resume Upload and Processing

Goal: Choose useful data from resume, and generate custom questions.

Frontend: The user uploads their resume as PDF.

- File upload component for resume selection.
- All other file formats are rejected.
- Upload status is shown to the user.

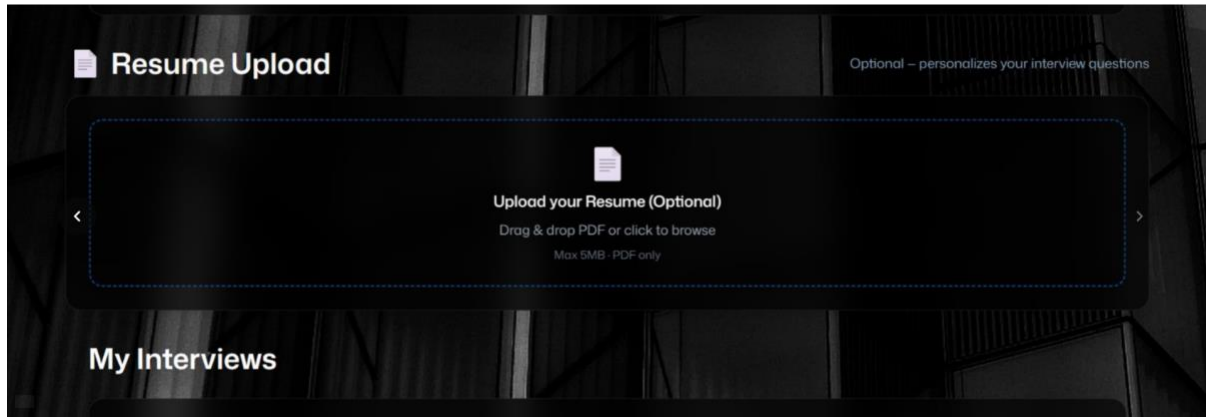


Fig 4.3 Resume Upload Page

Backend: Resume information is extracted.

- Resume is analyzed to extract skills and keywords.
- This data is stored temporarily.
- This data is used for AI generated questions.

4. AI-Based Question Generation

Goal: Use AI to generate interview questions relevant to the discussion.

Frontend: Questions are shown up during the interview, one at a time.

- The UI updates per question.
- User interaction is kept.

```

// Create interview object
const interview = {
  role,
  type,
  level,
  techstack: techstack.split(',').map((s: string) : string => s.trim()),
  questions,
  userId: currentUser.id, // Force attach user ID
  finalized: true,
  coverImage: getRandomInterviewCover(),
  createdAt: new Date().toISOString(),
};

await db.collection( collectionPath: 'interviews').add(interview);

return Response.json({ success: true }, { status: 200 });

} catch (error) {
  console.error('Interview generation error:', error);
  return Response.json(
    { success: false, error: (error as Error).message || 'Unknown error' },
    { status: 500 }
  );
}

```

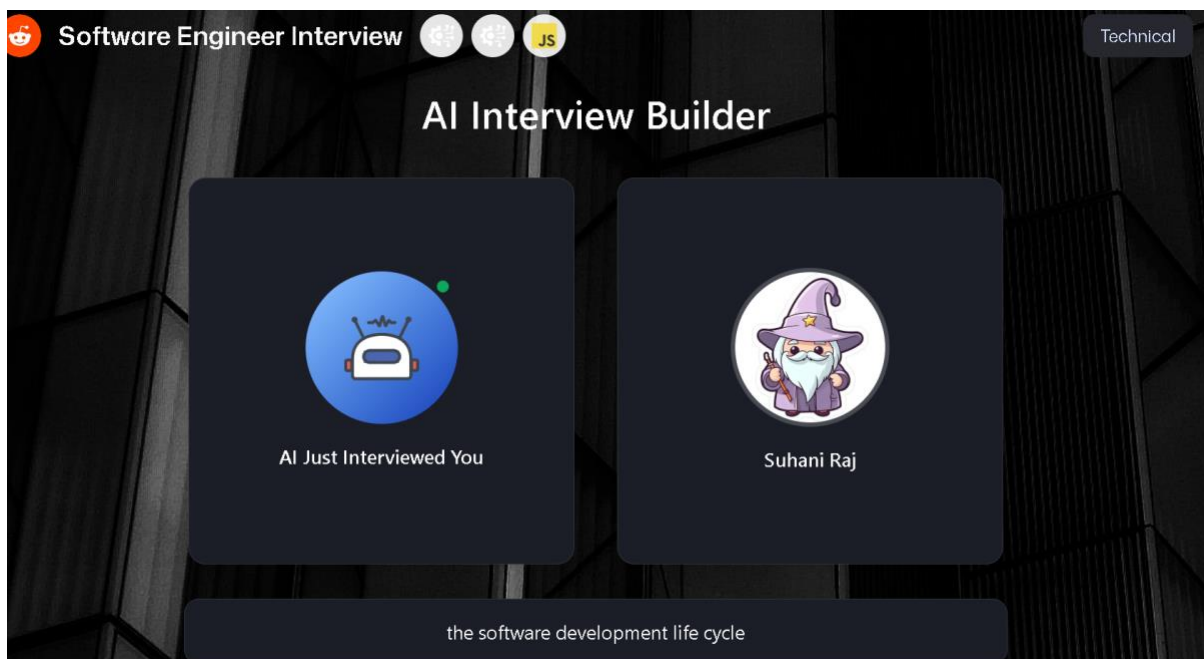


Fig 4.4 AI-Based Question Generation

Backend: We use AI APIs.

- Query (resume data or generic prompt) is passed to AI API.
- AI creates questions based on query.

- Questions are returned and viewed by the user.

5. Voice Input and Live Transcript

Goal: Add ability to answer questions by voice and real-time transcript display.

Frontend: The user answers questions by voice using microphone.

- Voice input is recorded through browser API's.
- Transcript is shown live.



Fig 4.5 Voice Input and Live Transcript

Backend: Speech is processed.

- Speech is transcribed into text using Speech API.
- The text is stored for analysis.
- Some minor error handling is applied for bad input.

6. Response Analysis and Feedback Generation

Goal : Analyze user answers and give informative answer.

Frontend: User sends answers one by one.

- Your answers are being saved and sent for grading
- You'll see the feedback at the very bottom of the interview.

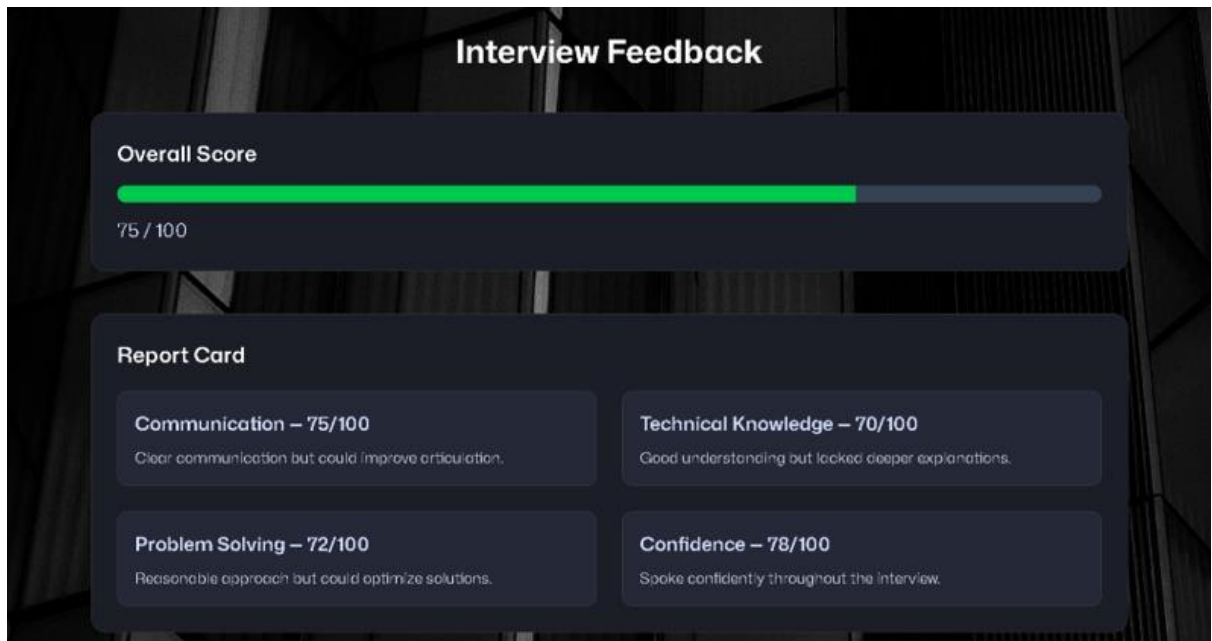


Fig 4.6 Feedback page

Backend: AI answers.

- User inputs are passed to the AI API.
- The AI has the say in the appropriateness and quality of the answer.
- The feedback is sent back as results.

7. Dashboard and Report Generation

Goal: Provide performance reports and downloadable reports.

Frontend: Dashboard shows user performance.

- Scores and feedback are shown as analytics.
- User can download report as PDF.

Backend: Data management and report creation.

- User data is stored in Firestore database.
- PDF is created through report libraries.
- Data is fetched and displayed in dashboard.

4.1.1 Use case Diagram

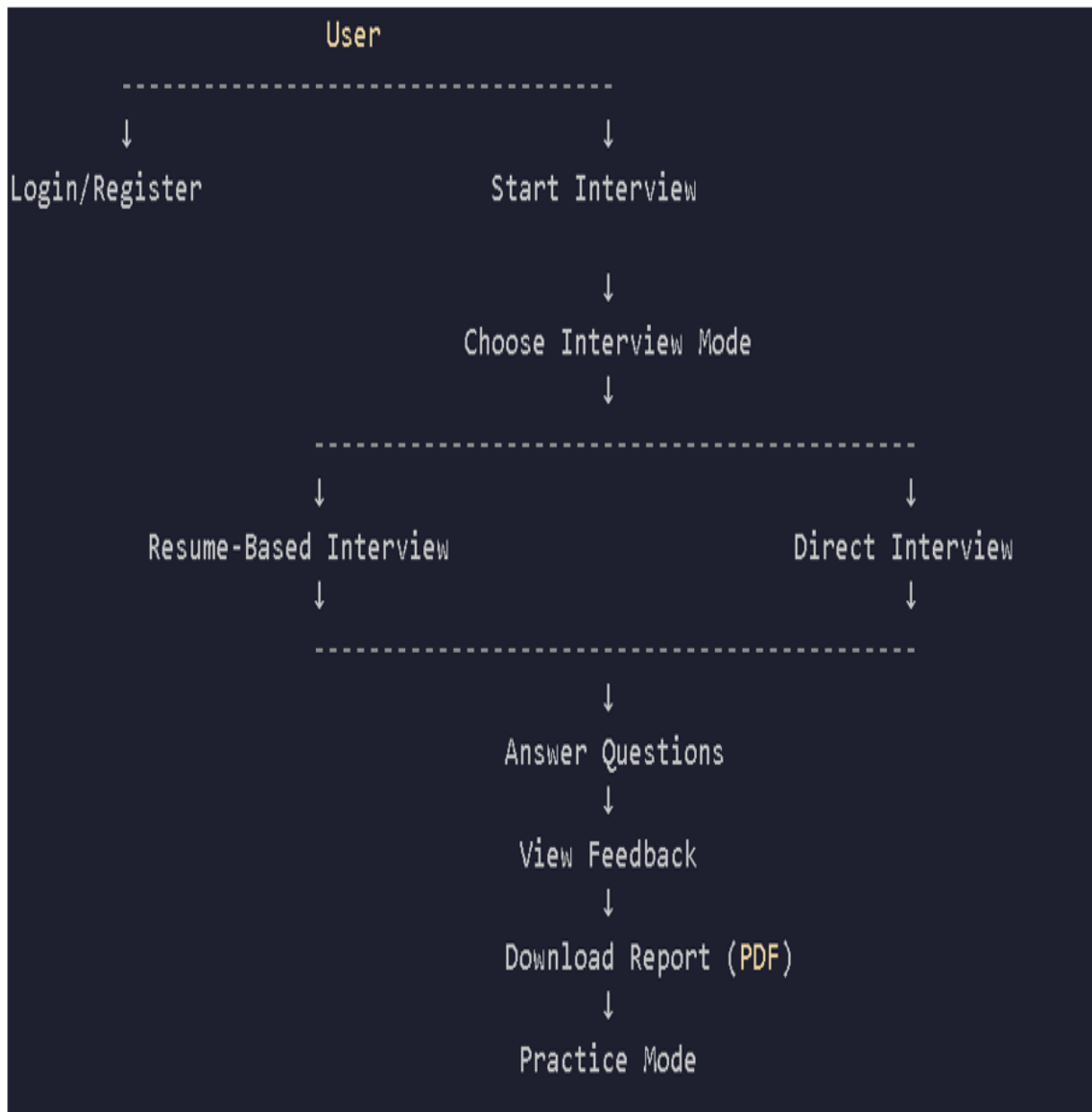


Fig 4.7 Use Case Diagram

4.1.2 Class Diagram

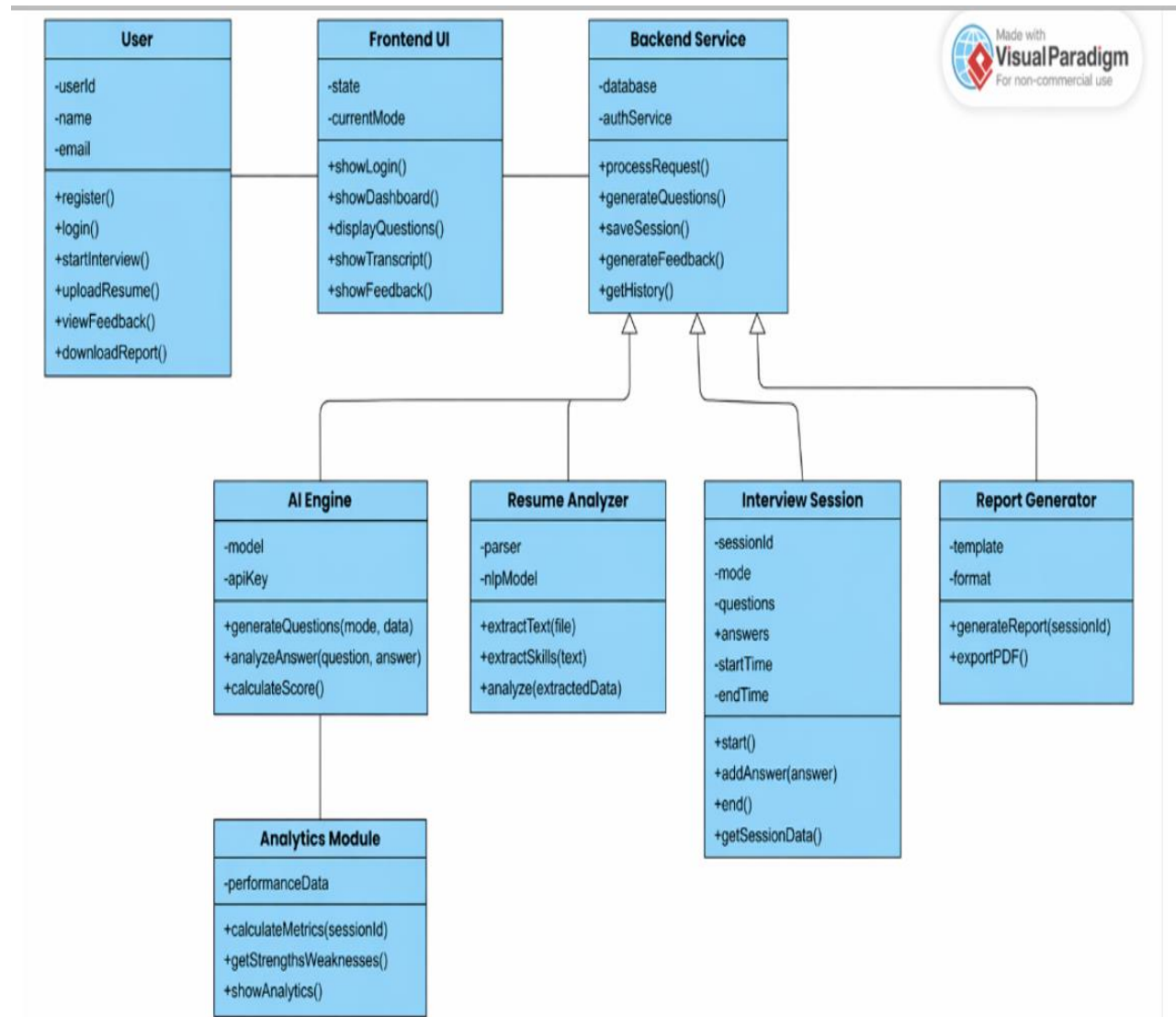


Fig 4.8 Class Diagram of Interview platform

4.2 Experimental Setup

The chosen tools and technology stack supported quick development, scalability, and integration of AI into the system.

4.2.1 Hardware Components

- **Processor:** Intel i5 or equivalent
- **RAM:** Minimum 8 GB
- **Storage:** 256 GB or higher
- **Input Devices:** Keyboard and mouse
- **Audio Input:** Microphone (for voice-based interview feature)

4.2.2 Software Components

- **Frontend:** Next.js (React framework)
- **Backend:** Firebase (Authentication and Firestore Database)
- **Programming Language:** TypeScript
- **AI Integration:** External AI APIs for question generation and response analysis
- **Speech Processing:** Browser Speech Recognition API
- **Development Tools:** Visual Studio Code
- **Version Control:** Git and GitHub
- **PDF Generation:** JavaScript-based PDF libraries

4.2.3 System Environment

- **Operating System:** Windows 10 / 11
- **Browser:** Google Chrome (recommended for best compatibility)
- **Internet Requirement:** Stable internet connection for API calls and database access

Tools/Technology	Component
Next.js	Frontend Framework
React.js	JavaScript Library
Firebase Authentication	Backend Service
Firebase Firestore	Database
AI APIs	Artificial Intelligence
Speech Recognition API	Browser API
PDF Generation Library	Utility Tool
VS Code	Development Tool
Git & GitHub	Version Control

Table 4.1 Tools and Technologies Used

4.3 Test Cases

4.3.1 Platform System Testing

The platform system testing was carried out in order to ensure the application proper operation. The modules of the system were examined separately for the proper functioning and for integration into other modules.

4.3.1.1 Login Module Testing

Input	Expected Output	Actual Output	Status
Valid email & password	User should log in successfully	Login successful	Pass
Invalid credentials	Error message should be displayed	Error message shown	Pass

Table 4.2 Login Module Testing

4.3.1.2 Resume Upload Module Testing

Input	Expected Output	Actual Output	Status
Upload valid PDF resume	Resume should be accepted and processed	Resume uploaded successfully	Pass
Upload invalid file format	Error message should be displayed	Error shown	Pass

Table 4.3 Resume Upload Module Testing

4.3.1.3 Interview Generation Testing

Input	Expected Output	Actual Output	Status
Resume based interview	Personalized questions generated	Questions generated	Pass

Direct interview	General questions generated	Questions generated	Pass
------------------	-----------------------------	---------------------	------

Table 4.4 Interview Generation Testing

4.3.1.4 Voice Input and Transcript Testing

Input	Expected Output	Actual Output	Status
User speaks answer	Speech converted to text	Transcript displayed	Pass
Noisy environment	Minor errors may occur	Slight inaccuracies observed	Pass

Table 4.5 Voice Input and Transcript Testing

4.3.1.5 Feedback Generation Testing

Input	Expected Output	Actual Output	Status
Submit interview responses	AI generates feedback	Dashboard working correctly	Pass

Table 4.6 Feedback Generation Testing

4.3.1.6 Dashboard and Analytics Testing

Input	Expected Output	Actual Output	Status
View dashboard	Performance analytics displayed	Dashboard working correctly	Pass

Table 4.7 Dashboard and Analytics Testing

4.3.1.7 PDF Report Generation Testing

Input	Expected Output	Actual Output	Status
Click download report	PDF file generated and downloaded	PDF downloaded	Pass

Table 4.8 PDF Report Generation Testing

4.4 Safety Measures

1. User **authentication** was used to allow only registered users to use the application. This is to protect user accounts from other users. firebase is used to login credentials.
2. User **data privacy** has been taken care of. data like resume etc. and interview response will be used is only for generating questions and feedback. no personal data is shared or misused by the system.
3. **Input validation** was also performed to prevent errors/misuse of the system. For example, only valid PDF file will be accepted to upload a resume.
4. **Safe content generation**, the model is deployed in a cautious manner in a way that only ask relevant and appropriate interview related question.
5. The system is intended to be operated in a **secure environment** involving correct flow of data and handling of api request, which may reduce risks of data leakage.

CHAPTER 5: RESULTS & DISCUSSION

5.1 Result Analysis

The created AI Interview Platform has been tested for all the features for performance and usability. The system can conduct interview in both resume based mode and direct interview mode giving flexibility to the user.

The response generation and feedback system was working as per plan, with AI generating questions and giving feedback in most cases. Voice to text feature was working nicely in normal condition with some error as per clarity of speech and background noise converting user speech into text. The analytics dashboard was helpful to visualize the performance of user to identify the strengths and weaknesses of user. The PDF generation feature was working as per plan to download the structured feedback.

5.1.1 Performance Table

Feature	Outcome	Result	Status
Question Generation	Relevant interview questions	Questions generated correctly	Pass
Resume Analysis	Personalized questions	Resume-based questions generated	Pass
Voice Input	Accurate transcript	Mostly accurate	Pass
Feedback Generation	Meaningful evaluation	Feedback generated successfully	Pass
Dashboard	Performance tracking	Analytics displayed correctly	Pass

PDF Export	Downloadable report	PDF generated successfully	Pass
------------	---------------------	----------------------------	------

Table 5.1 Performance Summary Table

5.1.2 AI Interview System Performance Result

Metric	Observation
Question Generation Accuracy	High
Response Evaluation Accuracy	Good
Average Response Time	~1–2 seconds
Resume Relevance	Maintained using extracted skills and keywords
Failure Handling	Proper fallback response when input data not found

Table 5.2 Interview System Performance Result

5.1.3 Interview Feature Performanc

Metric	Value
Voice-to-Text Accuracy	High
Feedback Quality	Good
Interview Completion Rate	High

Dashboard Accuracy	Accurate performance tracking
PDF Report Generation	Successful

Table 5.3 Interview Feature Performanc

5.1.4 Component vs Response Time

Component	Response Time
Question Generation	~1–2 seconds
Resume Processing	~2–3 seconds
Response Evaluation	~1–2 seconds
Voice-to-Text Conversion	~1 second
Dashboard Loading	~1 second

Table 5.4 Component vs Response Time

5.2 Comparison with Objectives

The results of the developed system are compared with the objectives mentioned in Chapter 1. After the comparison, it can be observed that the majority of the objectives have been achieved. The main goal of the project to create an AI based interview platform has been achieved. The system can simulate the real interview scenario. The candidate can be asked general questions generated by the system as well as resume-based personalized questions. The real-time voice input and live display of transcript has been achieved to enhance the user experience. The interview feedback can be downloaded in PDF. However, the system has a limitation in terms of accuracy of AI and voice recognition.

Objective	Expected Outcome	Achieved Outcome
AI-based Interview System	Simulate real interview	System working as expected
Personalized Questions	Resume-based questions	Implemented successfully
Real-time Interaction	Voice and transcript	Working with minor issues
Performance Tracking	Analytics dashboard	Implemented
Report Generation	Downloadable PDF	Working correctly

Table 5.5 Comparison with Objectives

5.3 Economic Analysis

Component	Cost
Next.js / React	Free
Firebase (Basic Plan)	Free / Low Cost
AI APIs	Limited free usage
Development Tools (VS Code)	Free

Table 5.6 Bill of Materials

5.3.1 Cost-Benefit Analysis

The total cost of system development is negligible as most of the tools used are free or have free versions. Compared to existing interview coaching platforms, this system is quite affordable for students.

5.3.2 Resource Utilization

The system is cost effective in terms of usage of resources like internet, computation and apis. As serverless backend is used it is scalable with usage.

Resource	Usage
Internet	Required for API calls and data sync
CPU	Handles frontend rendering and processing
Memory	Stores session data temporarily
Cloud Services	Firebase for backend operations

Table 5.7 Resource Utilization

5.4 Comparison with Existing Systems

Feature	Existing Systems	Proposed System
Question Type	Mostly general	General and Resume-based
Personalization	Limited	High
Feedback	Basic	Detailed AI-based
Voice Support	Rare	Available
Analytics	Limited	Dashboard available
Report Download	Not always available	PDF export included

Table 5.8 Comparison with Existing Systems

CHAPTER 6: CONCLUSION & FUTURE SCOPE

6.1 Summary of Achievements

In this project an AI interview platform was successfully built. This platform can be used by users to practice for interview in a simple and effective method. The platform mimics an authentic interview scenario, where users can answer questions and get feedback.

One of the key features of this project is the provision of two ways to conduct interview, resume based interview and direct interview. So the user can either be asked personalized questions based on their resume or just practice general interview questions.

The system can additionally automatically generate a question using an AI and user response. The system generates helpful feedback to the user response to enhance the performance.

Additional features live transcript display interview timer analytics dashboard and pdf report generation were introduced. These were interactive and useful to the user.

Designed and implemented a responsive and interactive user interface using React, providing smooth navigation, chatbot interaction, visual dashboards, and an overall improved user experience across devices.

6.2 Impact on society, Environmental sustainability Ethical issues and compliance

- **Democratizing Interview Prep:** India graduates about 1.5 million engineering graduates per year, most of whom attend colleges lacking the placement infrastructure, alumni network and financial resources to provide quality mock interview coaching. The AI Interview Agent offers the same coached, recorded, and evaluated mock interview for free to any student with a browser.

- **Pre-Interview Anxiety Alleviation:** Educational psychology research has consistently demonstrated that familiarity with a high-stress format lowers anxiety during the actual experience.

- **Paperless Feedback and Reporting:** All interview feedback, session transcripts, and performance reports are saved in Firestore and made available as downloadable PDF's.
- **No voice audio is stored:** We don't record or store the voice audio of the candidate, only the text transcript of the session is saved. Privacy concerns with voice biometric data are eliminated and the candidate can't accidentally leave a digital record of their spoken performance.
- **Openness about interaction with AI:** We are open to our users we are talking to them with AI interviewer instead of "tricking" our candidates into thinking that they are talking to a human. This is not only the right ethical thing to do, it will also establish the correct level of trust the users have in the feedback they receive.

6.3 Limitations

- **API Rate Limits:** The platform uses free-tier quotas from Vapi AI and Google Gemini. When the number of concurrent users is very high, these limits may be exceeded and voice sessions or feedback generation may fail. This is currently the biggest operational bottleneck to scaling the platform.
- **English language only:** All voice interactions, question generating and feedback are in English.
- **No Code Practice:** There is no code editor and no auto-evaluation of test cases. It's all verbal interview practice. Candidates can't prepare for coding rounds, where they need to write and run code, as this here.
- **Practice Mode Does Not Score:** The text-based Practice Mode does not produce a scored feedback report at the end of the session with question delivery only. Candidates who use the Practice Mode do not have the ability to compare their score from the text-based question delivery to their voice session in the Analytics Dashboard.

6.4 Future Work

- **Multi-Language Support:** Robust support for Hindi and other regional languages can help us reach more candidates across India. We can choose a voice API that supports multiple languages and configure Gemini with language specific prompts so that candidates who prefer to think and express technical concepts in their native language can feel comfortable in using the platform.
- **Built-in Code Editor for Technical Rounds:** A built-in code editor with syntax highlighting, test case execution, and AI-based code review would allow the platform to encompass coding interview rounds.
- **Peer-Review/Community Features:** the capacity to share anonymized transcripts of the session, and have it peer or mentor review to add a human review layer on top of the AI.
- **Emotion and Confidence Detection:** If we were to combine speech emotion recognition, to identify nervousness, hesitation, or overconfidence in vocal cues , with Gemini's content analysis, it would provide a soft skills assessment component to the feedback. This would allow candidates to gain a more rounded understanding of how they present themselves in an interview.
- **Offline Practice Mode with Local LLM:** While students in areas with limited internet connectivity will be served by the core Q&A and feedback functionality, a light-weight LLM that runs locally could potentially be integrated into a desktop version of the application, allowing practice without an internet connection.
- **Deployment at Institutional Level for Colleges:** An institutional level deployed multi-tenant version of the platform can be used by placement cells at colleges to allocate mock interview sessions to their students, monitor cohort level performance and identify topics that need to be reinforced in the classroom before the placement season.

REFERENCES

- [1] Vapi AI. Vapi AI Developer Documentation. [Online]. Available: <https://docs.vapi.ai>.
- [2] B. Kotsiantis, “Artificial Intelligence in Education: A Review,” *International Journal of Emerging Technologies in Learning*, vol. 14, no. 9, pp. 56–72, 2019.
- [3] Official documentation of Next.js was used for understanding frontend development, routing, and performance optimization in web applications. [Online]. Available: <https://nextjs.org/docs>.
- [4] Firebase Documentation, Google was consulted for user authentication and real-time database usage in our system. [Online]. Available: <https://firebase.google.com/docs>
- [5] Consulted AI API documentation and examples for adding question generation and feedback in the system. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API.
- [6] M. McTear, Z. Callejas, and D. Griol, *The Conversational Interface: Talking to Smart Devices*, Springer, 2016.

APPENDICES

Appendix A: Key Code Snippets

```
import { google } from '@ai-sdk/google';
import { generateText } from 'ai';
import { db } from "@firebase/admin";
import { getCurrentUser } from "@lib/actions/auth.action"; // ✅ add this
import { getRandomInterviewCover } from '@lib/utils';

export async function POST(request: Request) : Promise<Response> { Show usages 👤 RajSuhani *
  try {
    const body : any = await request.json();

    // Input validation (remove userId from required fields)
    const { type, role, level, techstack, amount } = body || {};
    if (!type || !role || !level || !techstack || !amount) {
      return Response.json(
        { success: false, error: 'Missing required fields' },
        { status: 400 }
      );
    }

    // Get logged in user
    const currentUser : User | null = await getCurrentUser();
    if (!currentUser) {
      return Response.json(
        { success: false, error: 'User not authenticated' },
```

```
        { status: 401 }
      );
    }

    // Generate interview questions
    const { text: questionsText } = await generateText({
      model: google( modelId: 'gemini-2.0-flash-001'),
      prompt: `Prepare questions for a job interview.
The job role is ${role}.
The job experience level is ${level}.
The tech stack used in the job is: ${techstack}.
The focus between behavioural and technical questions should lean towards: ${type}.
The amount of questions required is: ${amount}.
Please return only the questions, without any additional text.
Return like: ["Question 1", "Question 2", "Question 3"]`,
    });
```

```

// Create interview object
const interview = {
  role,
  type,
  level,
  techstack: techstack.split(',').map((s: string) : string => s.trim()),
  questions,
  userId: currentUser.id, // Force attach user ID
  finalized: true,
  coverImage: getRandomInterviewCover(),
  createdAt: new Date().toISOString(),
};

await db.collection( collectionPath: 'interviews').add(interview);

return Response.json({ success: true }, { status: 200 });

} catch (error) {
  console.error('Interview generation error:', error);
  return Response.json(
    { success: false, error: (error as Error).message || 'Unknown error' },
    { status: 500 }
  );
}

```

Appendix B: System Architecture Overview

B.1 Architecture

- **Client Layer:** Renders all the UI, manages the Firebase Authentication session, Vapi AI WebRTC voice stream, and displays real-time transcript in React state.
- **Voice Layer:** The Vapi AI SDK controls the WebRTC audio channel from the browser to the Vapi cloud, which does speech to text, sends conversation turns to the configured AI assistant, and streams text to speech audio back to the browser.
- **API Layer:** The API routes in Next.js (app/api/) provide a secure server-side interface between the client and external AI services.
- **AI Layer:** Google Gemini API (via @ai-sdk/google) is used in the API routes to generate post interview feedback and resume based question personalization.
- **Data Layer:** Profile data, interview session metadata and generated feedback in Firebase FirestoreUser identity in Firebase Authentication with Google OAuth Firestore security rules to implement per-user isolation of all client reads/writes.

B.2 Data Flow — Interview Session

1. User logs in with Firebase Authentication. Session token being stored on the client-side.
2. User uploads a resume or choose interview type. If a resume was uploaded, client POSTs to `/api/generate-questions`, which calls Gemini and returns a custom question set.
3. A new interview session document with status 'started' is created in Firestore. The Vapi AI session is started with the question set injected in the assistant prompt.
4. During a session, all voice audio gets streamed to the Vapi cloud for transcription, Vapi does not store any audio.
5. At session end, the entire transcript is POSTed to `/api/feedback` which calls Gemini to create a structured feedback object, and writes it to Firestore.
6. The AI evaluation is viewable and the user can download the PDF report and view the Analytics Dashboard.

ANNEXURES

Annexure I: Plagiarism Report Summary

Annexure II: PO / PSO Mapping

PO/PSO	Description	Project Mapping
PO1	Engineering Knowledge	Used Next.js 15, React, TypeScript, Firebase Firestore, REST APIs, and Vapi AI concepts
PO2	Problem Analysis	Recognized need for easier voicebased interview preparation
PO3	Design Solution	Designed AI-based Interview Platform
PO4	Investigation	Tested models and evaluated performance
PO5	Modern Tools	Used Next.js 15, Firebase, Vapi AI SDK, Google Gemini API, Tailwind CSS and GitHub for version control
PO6	Society Impact	Improves accessibility for students
PO7	Environment	Reduces paper usage
PO8	Ethics	Fixed bias in eval prompts, added more strict Firestore security rules to protect user data
PO9	Team Work	Collaborative development
PO10	Communication	Report writing and UI design
PO11	Project Management	Followed timeline and phases
PO12	Lifelong Learning	Explored latest AI tools

Annexure III: Outcome of the Project



Fig 1: sign-in/ sign-up page

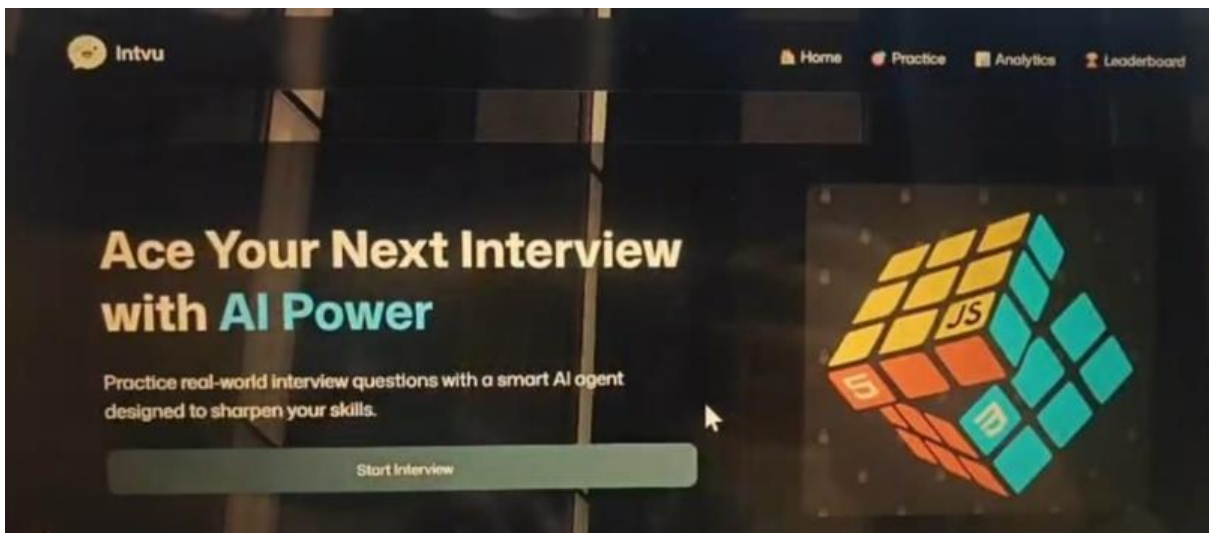


Fig 2: Home page

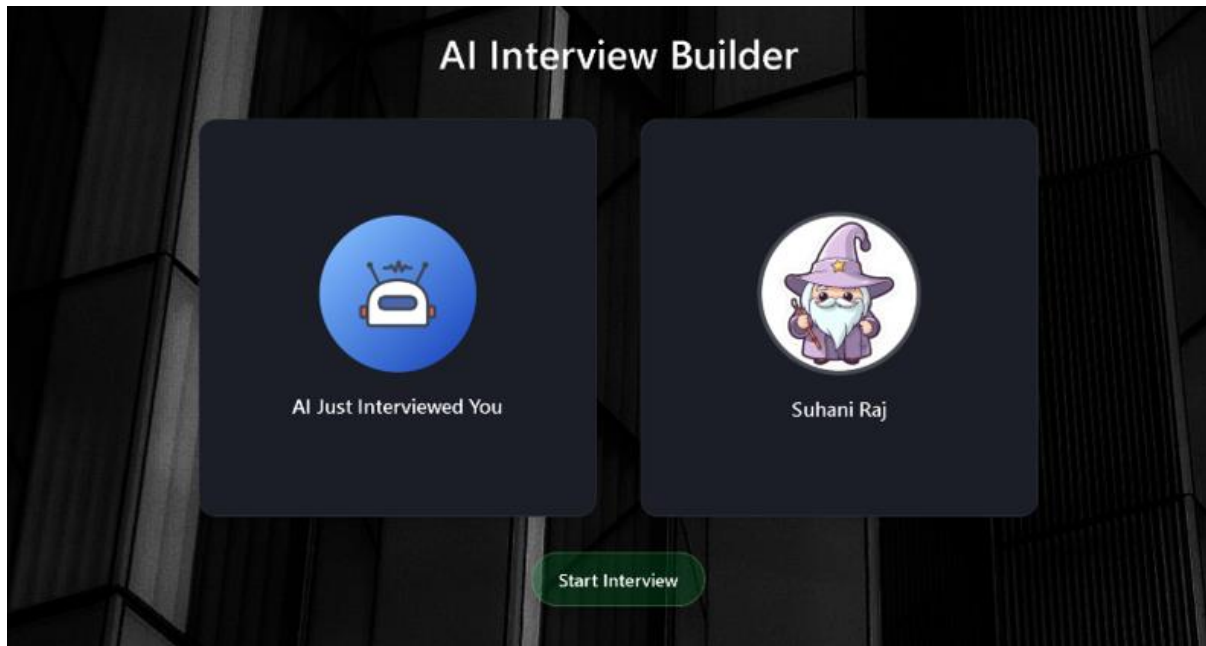


Fig 3: Main interface of the AI Interview

Annexure IV: Sustainability Statement

IV.1 Environmental Impact

- No physical infrastructure: No need for physical premises, print material, or computer hardware. Travel emissions, electricity usage of the physical coaching centre, and paper consumption by printing study material would have been generated in case the mock interview was held in a single session on the platform.
- Paperless Feedback: All reports, transcripts and analytics are retained and delivered electronically. PDF export is an electronic-only document and not a traditional print document.

IV.2 Social Impact

- Confidence and employability: repeated practice of realistic interview scenarios creates familiarity with an interview format, lessens pre-interview anxiety and enhances verbal skills, all of which have a direct positive influence on student placement results.

- Inclusive design: The Practice mode page ensures that those students with a hearing or speech impairment, limited bandwidth, or older devices have access to everything the platform has to offer in terms of preparation, making it useful for all user scenarios.

Annexure V: Team Roles and Contributions

Name	Contribution
Suhani Raj	Next.js 15 App Router architecture; Firebase Auth (Google OAuth) and Firestore database design; Vapi AI SDK voice integration; Gemini API integration for feedback and resume analysis; GitHub repository management; general project management
Priyanshi Rana	Next.js server-side API routes. Validating request and error handling. Parsed the response from Gemini API. Analytics Dashboard with category wise and time-series charts from Firestore data. Interview Timer Component with progress bar and countdown.
Dhruv Agarwal	UI/UX design and implementation using Tailwind CSS and ui components with TypeScript; Live Transcript page with live Vapi event listeners; Practice Mode text based Q&A page; PDF download functionality; end-to-end testing in multiple browsers and screen sizes

